ARTI2001 · M1 T3 分类（Classification）知识点全集

从线性得分到概率与决策 · 涵盖 28 页课件 + Tutorial 三题 + Practical 全部内容 + 阅读材料 + 15 项数学推导
标记说明：易错 高频失分点 重点 考试重点

第一部分 · 课件（Slides 1–28）

第 1–3 页：开篇与路线图

标题 *From Linear Scores to Probabilities and Decisions*（从线性得分到概率与决策）就是整个 topic 的主线。第 2 页 Acknowledgement of Country 不考。

路线图：回归 → 分类 → 线性得分 z → 感知机 → 逻辑回归 → 损失与梯度 → tutorial / practical

> **整份课件的核心一句话**：感知机和逻辑回归用的是**同一个线性得分 z**，区别只在于 z 之后怎么处理（输出变换、损失函数、更新信号）。

第 4 页：回归与分类共用一条流水线

共同点：两者都是**监督学习**，训练时都给 x 和已知的 y，整条流水线（特征 → 模型 → 预测 → 损失学习）一样。

不同点：只在于**目标变量的类型**，以及由此带来的损失函数不同。

	回归 Regression	分类 Classification
预测什么	连续数值	离散类别
例子	房价、气温	垃圾邮件 / 非垃圾邮件
输出	一个实数	类别标签，或类别的概率
常用损失	均方误差 MSE	二元交叉熵
问的问题	**how much?**（多少）	**which class?**（哪一类）

分类的输出可以是标签，也可以是概率：感知机直接给标签（+1/−1）；逻辑回归先给概率再靠阈值转成标签。

第 5 页：分类从"表示（representation）"开始

模型只会算数，任何对象都要先转成**特征向量 x**。课件例子：简化版词袋 `x = [开心词数量, 难过词数量]`

句子	特征 x	句子	特征 x
I love sunny days	[2, 0]	I am sad and tired	[0, 2]
I feel down today	[0, 1]	What a sunny day	[1, 0]
The weather is great	[1, 0]	A neutral message	[0, 0]

全 0 也是合法的特征向量。原则：**特征的顺序对每个样本都必须一致**——否则同一个权重在不同样本上代表不同含义。

记号约定（贯穿全课件）

符号	含义	符号	含义
x	特征向量	m	训练样本的**数量**（不是维度！）
y	已知的真实类别标签（ground truth）	$\hat{y}$	模型**预测**的类别
$(x^{(i)}, y^{(i)})$：第 i 个带标签的训练样本			

> 易错 **标签约定与模型绑定**：感知机用 **−1 / +1**（更新规则要用标签符号做乘法）；逻辑回归用 **0 / 1**（输出是概率，落在 [0,1]）。一旦选定不能中途换——这是最常见的失分点。

第 6 页：线性得分是两个模型的共同内核

$$z = w^T x + b = w_1 x_1 + w_2 x_2 + \cdots + w_d x_d + b$$

课件原话：*both the perceptron and logistic regression begin with the same linear score*。$w^T x$ 是点积，结果是标量。

参数	作用
w（权重）	决定哪些特征重要。正权重 → 该特征越大 z 越大（推向正类）；负权重 → 推向负类；绝对值越大影响力越强，接近 0 则被忽略。**改变权重会让边界旋转（rotate）**
b（偏置）	**平移（shift）边界，但不改变朝向**

重点 **"w 转，b 移"**。理解方式：$z = 0$ 即 $w_1x_1 + w_2x_2 + b = 0$ 是一条直线，w 决定斜率（方向），b 决定截距（位置）。

- 每个输入都拿到一个得分 z；**z 的符号决定它落在边界哪一侧**
- **$z = 0$ 就是决策边界本身**；落在线上的点最"没把握"

易错 **z 本身不是概率**。z 的取值范围是整个实数轴，可以是 −7.3 或 +150。想要概率必须再套 sigmoid。

第 7 页：一条边界把特征空间切成两半（几何视角）

读图五条：① 点是带标签的训练样本；② 线是 $z = 0$ 的位置；③ 一侧 $z > 0$、另一侧 $z < 0$；④ 改 w 线旋转、改 b 线平移；⑤ $|z|$ 越大点离边界越"远"，模型越有把握。

重点 **$|z|$ 是"得分距离"，不是普通的几何距离**（a score distance rather than an ordinary geometric distance）。真正的垂直距离是：

$$\text{距离} = |z| / \|w\|，\text{其中 } \|w\| = \sqrt{(w_1^2 + w_2^2)}$$

为什么？ 把 w 和 b 同时乘以 10，边界那条线完全没动（同一条直线），但每个点的 z 都变成 10 倍。除以 $\|w\|$ 才能消掉这个缩放。
实用结论：同一个模型内部比较 $|z|$ 有意义；两个不同模型的 z 值直接比大小没有意义。

第 8 页：感知机——用符号做预测

本页把得分记作 **s**（与 z 同物，$s = w^Tx + b$）。

$$\text{预测：} \hat{y} = \text{sign}(s) \quad | \quad \text{判断对错：signed score} = y \cdot s$$

没有 sigmoid、没有概率，直接看符号——这就是感知机与逻辑回归的分岔口。

$y \cdot s$	含义	动作
$y \cdot s > 0$	y 和 s 同号，预测在正确一侧	不更新
$y \cdot s < 0$	y 和 s 异号，分错了	更新
$y \cdot s = 0$	样本正好落在决策边界上（平局 tie）	**本课程约定：仍要更新**

易错 **本课程的更新触发条件是 $y \cdot s \leqslant 0$（小于等于）**。别的教材可能写 < 0，做题按 $\leqslant$ 来。
另：**标签用 −1/+1 的原因**——乘积的符号能直接编码"对/错"。若用 0/1，$y = 0$ 会把乘积压成 0，这个机制就失效了。

课件例子表

i	得分 s	标签 y	$y \cdot s$	要更新吗
1	+1.3	+1	+1.3	否
2	−0.8	−1	+0.8	否（两负相乘得正！）
3	−0.4	+1	−0.4	**是**
4	0	−1	0	**是**（$\leqslant$ 规则）

第 9 页：带符号得分定义了感知机损失

$$L_i = \max(0, -y_i s_i) \quad \text{（感知机准则 perceptron criterion）}$$

情况	$-y \cdot s$	损失 L	含义
$y \cdot s > 0$（分对）	负数	**0**	正确一侧，零损失
$y \cdot s < 0$（分错）	正数	正数	错误一侧，有损失
$y \cdot s = 0$（平局）	0	**0**	正好在边界上

max(0, ·) 的作用：防止损失为负。没有它，分得特别对的点会给出很大的负损失，模型会一味把已分对的点推得更远而不去修正错的点。有了 max，**分对的样本对损失零贡献**——感知机只关心犯错的样本。

易错 **损失函数与更新规则在 $y \cdot s = 0$ 处不一致**：损失 $L = \max(0, 0) = 0$，但更新规则 $y \cdot s \leqslant 0$ 仍要更新。**做题以更新规则为准**，不要因为损失是 0 就跳过这个样本。

- 对分错的点，$L = -y \cdot s = |s|$：**错得越离谱损失越大，且是线性增长**（不像 MSE 平方增长）
- 取值范围 **$[0, +\infty)$**，永远非负，下界 0 可取到
- 导数：分错区域 $L = -y \cdot s$，对 w 求导得 $-y \cdot x$，梯度下降方向即 **$w \leftarrow w + y \cdot x$**——更新规则是从损失推出来的

第 10 页：感知机的训练是逐样本顺序进行的

核心：**每个样本用的都是前一个样本刚更新出来的参数**。

1. **初始化**：w 和 b 通常取全零或很小的值
2. **计算得分**：$s_i = w^T x_i + b$，预测 $\text{sign}(s_i)$
3. **检查 $y_i s_i$**：正 → 保持不变；零或负 → 更新
4. **更新并带着走**：**$w \leftarrow w + \eta\, y_i\, x_i$，$b \leftarrow b + \eta\, y_i$**，立刻用新参数处理下一个样本

> **为什么这条规则有效**：假设 y = +1 但分错了，更新后新得分 s_new = s_old + η(‖x‖² + 1)，因为 ‖x‖² + 1 恒正，所以 **s 一定变大**，朝正类方向修正。y = −1 时同理 s 变小。
> b 的更新式就是把 w 的公式里的 x 换成 1——偏置可看作一个恒等于 1 的虚拟特征的权重。

术语	定义
步 step	对**一个样本**的一次更新机会（不管最后有没有真的更新）
轮 epoch	把**每个样本都处理一遍**
顺序性 sequential	参数立刻生效，下一个样本用更新后的参数

> 易错 **改变样本顺序会改变中间参数值和学习路径**。这是感知机与"批量"方法的重要区别。最常见的错误：算完第 1 个样本的更新后，第 2 个样本还在用旧的 w。

第 11 页：一次更新就能把样本推过边界（手算例）

已知：w = [−0.5, 0.2]，b = 0，x = [2, 0]，y = +1，η = 0.5

步骤	计算
① 算得分	s = (−0.5)(2) + (0.2)(0) + 0 = **−1**
② 判断	y · s = (+1)(−1) = −1 ⩽ 0 → **分错，需更新**
③ 更新 w	w ← [−0.5, 0.2] + 0.5(+1)[2, 0] = **[0.5, 0.2]**
③ 更新 b	b ← 0 + 0.5(+1) = **0.5**
④ 验证	s_new = (0.5)(2) + (0.2)(0) + 0.5 = **1.5**，从 −1 → +1.5，翻到正确一侧

> 重点 **w_2 没有变**（0.2 还是 0.2），因为更新量 η · y · x_2 中 $x_2 = 0$。**特征值为 0 的那一维，权重不会被更新。**

三个关键词：**η** 控制修正的**幅度**；**y · x** 决定更新的**方向**；**carry** 更新后参数立即用于下一个样本。η 太小翻不过边界，太大会矫枉过正。

第 12 页：感知机能保证什么、不能保证什么

#	内容
①	**感知机收敛定理**：若训练数据**线性可分**，算法**只会犯有限次错误**，之后能找到一条分隔边界。注意：保证的是"错误次数有限"，**不保证快、也不保证找到最优的那条线**（追求最大间隔的是 SVM）。经典反例是 XOR。
②	**类别重叠时更新可能永不停止**，参数在几组值之间来回循环（cycle）。实践中必须人为设停止条件。
③	顺序更新使**中间路径依赖于样本顺序**。
④	学习率只**缩放修正幅度，不产生概率**（scales each correction but does not create probabilities）。
⑤	输出是**硬类别判定**（hard class decision），不是**校准过的概率**。

> 易错 **一个很大的正得分不能解释成"90% 的概率"**。s = 9 不代表 90% 把握——s 的尺度取决于 w 的大小（w、b 同乘 10，边界不动但得分全变 10 倍）。**这五条局限正是引出逻辑回归的动机。**

第 13 页：逻辑回归把同一个得分"抹平"成概率

$$p = \sigma(z) = 1 / (1 + e^{-z})$$

sigmoid（logistic 函数）把 (−∞, +∞) 压缩到 **(0, 1)**，输出解释为**模型估计的 y = 1 的概率**。

z	−6	−4	−2	**0**	2	4	6
p	0.002	0.018	0.119	**0.5**	0.881	0.982	0.998

四条性质

1. **z = 0 → p = 0.5**（$1/(1+e^0) = 1/2$）。决策边界 z = 0 对应概率恰好 0.5，和感知机是同一条线
2. **z > 0 → p > 0.5；z < 0 → p < 0.5**。"取 z 的符号"与"看 p 是否超过 0.5"是**等价判断**
3. $z \to \pm\infty$ 时 $p \to 1 / 0$，但**永远取不到 0 或 1**（e^{-z} 恒大于 0）。正因如此交叉熵里的 log 不会爆掉
4. **对称性**：σ(−z) = 1 − σ(z)。验证：σ(−2) = 0.119、σ(2) = 0.881，相加为 1

重点 **The sigmoid changes scores into probabilities; it does not change the linear boundary family.**
sigmoid 改变的是输出的**含义，不改变决策边界仍是线性的**这一事实。因为 $p > 0.5$ 等价于 $z > 0$，而 $z = 0$ 依然是直线。**逻辑回归同样解决不了 XOR。**

第 14 页：阶跃函数 vs. Sigmoid

	感知机的阶跃 step	逻辑回归的 sigmoid
图形	在边界处突然跳变	随 z 平滑变化
输出	**硬标签**（class only）	**概率**（smooth output）
相邻的 z	可能得到完全不同的输出	得到相近的概率
导数	几乎处处为 0，边界处不存在	处处存在且为正

z	−3	−2	−1	**0**	1	2	3
p	0.047	0.119	0.269	**0.5**	0.731	0.881	0.953

阈值化 thresholding

$p \geqslant 0.5 \rightarrow$ 预测类别 1；否则预测类别 0

- **注意等号在哪边**：课件写 $p \geqslant 0.5$ 判为类别 1，正好等于 0.5 时归正类
- **阈值化在算出概率之后才做**。顺序是 **z → p → 阈值 → 标签**。所以阈值可以改（医疗筛查可调到 0.2 减少漏诊），而感知机没有这个旋钮

重点 **平滑为什么重要**：小的参数变化带来概率与损失的渐变，使**基于梯度的优化成为可能**。阶跃函数除边界外导数处处为 0、边界处不可导，梯度收不到任何信号——这正是感知机要用特设更新规则而非标准梯度下降的原因。**平滑不是为了好看，是为了让学习成为可能。**

第 15 页：在 p = 0.5 处，逻辑回归仍然是线性的

推导链：① $\sigma(0) = 0.5$ → ② 因 sigmoid **严格单调递增**（因而可逆），$p = 0.5 \Leftrightarrow z = 0$ → ③ 两个特征时边界满足 $\mathbf{w_1x_1 + w_2x_2 + b = 0}$，这是一条直线方程。

斜截式（$w_2 \neq 0$ 时）：$x_2 = -(w_1/w_2) \cdot x_1 - b/w_2$ → 斜率 $= -w_1/w_2$，纵截距 $= -b/w_2$

易错 前提 $\mathbf{w_2 \neq 0}$。若 $w_2 = 0$，方程变成 $x_1 = -b/w_1$，这是一条**垂直线**，斜率无穷大，斜截式表示不了，必须单独处理。

判类流程：画出线 → 对任意测试点算 z → 看符号。$z > 0 \rightarrow p > 0.5 \rightarrow$ 类别 1；$z < 0 \rightarrow$ 类别 0。**做题时不用真的算 sigmoid，算 z 的符号就够了**，只有问"概率是多少"时才代入 sigmoid。

第 16 页：二元交叉熵惩罚"自信的错误"

标签用 $\mathbf{y \in \{0, 1\}}$（不是感知机的 −1/+1）。

$$L(y, p) = -[\, y \cdot \log(p) + (1-y) \cdot \log(1-p) \,]$$

关键技巧：它其实是一个"开关"。因为 y 只能取 0 或 1，每次只有一项存活：
$y = 1 \rightarrow L = -\log(p)$ ｜ **$y = 0 \rightarrow L = -\log(1-p)$**
做题先看 y 是几，直接套对应的半个公式。

y = 1 时的 p	0.99	0.5	0.1	0.01
L = −log(p)	0.01	0.69	2.30	**4.61**
评价	又对又自信	完全不确定	错了	**自信地错了**

惩罚是**非对称、爆炸式**增长的：$p \rightarrow 0$ 时 $-\log(p) \rightarrow +\infty$。对比感知机损失只是线性增长。（正因 sigmoid 取不到 0 和 1，log 才不会真的无穷大，训练不会崩。）

性质	含义
p near y → small loss	预测概率贴近真实标签，损失就小
smooth（平滑）	支持梯度下降
probabilistic（概率化）	奖励**校准过的**置信度（calibrated confidence）——模型说 70% 的那些样本，实际约真有 70% 是正类。这正是感知机做不到的

训练最小化的是所有样本损失的平均值（average loss over all examples），写目标函数时记得 $(1/m) \cdot \Sigma$。

第 17 页：梯度由 p − y 驱动

sigmoid 和交叉熵单独看都复杂，组合求导后全部消掉，只剩一个减法：

$$\partial L_i/\partial z_i = \mathbf{p_i - y_i} \quad \text{（signed probability error 带符号的概率误差）}$$

$$\partial L_i/\partial w = (p_i - y_i)\cdot x_i \qquad \partial L_i/\partial b = p_i - y_i$$

（因为 $\partial z/\partial w = x$，$\partial z/\partial b = 1$。）梯度下降沿**负梯度**方向走：$-\eta\nabla J$。

情况	含义	该怎么办
$p - y < 0$	概率**低于**目标	**提高**得分
$p - y > 0$	概率**高于**目标	**降低**得分

例：$y = 1$、$p = 0.2 \to p - y = -0.8$，概率报低了，要把 z 推高。$y = 0$、$p = 0.9 \to +0.9$，报高了，要压低。

重点 **与感知机的关键区别**：感知机不管错多少，更新量都是固定的 ηyx；逻辑回归**按错误的严重程度按比例调整**，接近正确时自动放缓。

- **特征值为 0 → 对该权重的直接贡献为 0**（梯度里乘了 x_i）——与第 11 页感知机性质相同
- **偏置的梯度就是 $p - y$ 本身**，不乘任何特征。所以**即使所有特征都是 0，b 依然会被更新**
- **批量更新前先取平均**（average contributions before a batch update）

第 18 页：批量更新与随机更新用的是同一个思路

两者用**完全相同的单样本梯度贡献** $(p_i - y_i)x_i$，区别只在于**更新参数之前让多少个样本参与**。

	批量 Batch	随机 Stochastic (SGD)
每次更新用几个样本	全部 m 个	一个
信号	平均梯度	单个样本的贡献
路径	更**平滑**（smoother）	更**嘈杂**（noisier）
适用于	稳定的 epoch 训练	**在线学习 / 大数据**

批量：梯度平滑且**确定的（deterministic）**，但每次更新都要处理整个数据集，数据量大时很慢。**随机**：更新频繁、支持**在线学习**，但路径嘈杂、会在最优点附近抖动。

小批量 Mini-batch：每次取一小组样本求平均再更新，**在梯度稳定性与计算效率之间取得平衡**，实践中几乎所有深度学习训练都用它。

批量（m 个）—— 小批量（如 32 个）—— 随机（1 个）
← 更稳定、更慢 ・ 更嘈杂、更快 →

共同点：三者都沿负梯度方向走（both follow the negative gradient），优化同一个目标函数，区别只是梯度估计得多准。

第 19 页：相同的边界族，不同的学习信号

共同点：都从 $z = w^Tx + b$ 出发，都用 $z = 0$ 作为标准决策边界，属于**同一个线性边界族**——能画出的边界形状完全一样。

	感知机	逻辑回归
定位	**硬预测** hard prediction	**概率模型** probability model
标签约定	**−1 和 +1**	**0 和 1**
输出	**sign(z)**	**σ(z)**
何时更新	只在犯错或平局时（$y \cdot z \leqslant 0$）	只要梯度不为零就更新
学习信号	**$y \cdot z$**（带符号得分）	**$p - y$**（概率误差）
概率	**没有**概率估计	阈值化**之前**概率是可用的

四条最容易考的差异

1. **更新时机**：感知机分对了就完全不动；逻辑回归只要 $p \neq y$ 就有非零梯度，而 **p 永远取不到 0 和 1**，所以**每个样本每次都在贡献梯度**（只是很小）。这是最本质的行为差异
2. **学习信号性质**：$y \cdot z$ 只关心符号（二元判断）；$p - y$ 是**连续量**，既带方向又带大小
3. **概率**："probability before thresholding"——逻辑回归**能做感知机能做的一切，还多给一个概率**
4. **标签约定绑定模型**：−1/+1 配感知机（乘法判对错），0/1 配逻辑回归（对应概率区间）

Study check：这个差异什么时候会改变建模决策？
当你需要的不只是"哪一类"，而是"多有把握"时，就必须选逻辑回归。比如：要排序（挑出最可能是欺诈的 100 笔交易）、要调整阈值（医疗筛查宁可误报不可漏诊）、要把置信度传给下游系统、或不同类别代价不同时的决策。反之，只需类别标签且数据线性可分时，感知机更简单更快。

第 20 页：检查点——把五个量串起来

#	量	内容
①	**表示** Representation	把原始对象转成数值特征向量 x
②	**线性得分** Linear score	$z = w^Tx + b$（两个模型共用）
③	**概率** Probability	$p = \sigma(z)$。**只有逻辑回归有，感知机跳过**
④	**预测类别** Predicted class	感知机 sign(z)；逻辑回归把 p 在 0.5 处阈值化
⑤	**学习信号** Learning signal	感知机：$y \cdot z \leqslant 0$ 才更新；逻辑回归：用 $p - y$ 构造梯度

易错 **两个高频计算错误**（课件点名）：
① **把 z 当成概率**——z 可以是 −7 或 +150，必须先过 sigmoid。
② **在逻辑回归公式里用感知机的标签**——把 $y = -1$ 代进交叉熵会得到 $-[-\log p + 2\log(1-p)]$，毫无意义。逻辑回归必须用 0/1。

把链背成一句话：表示 → 得分 →（概率）→ 类别 → 学习信号。看到题目先定位：问的是链上哪一步？用的是哪个模型？标签是哪套约定？

第 21 页：第一部分完成——分类基础

五条核心要点

1. 分类预测类别，但仍然从数值特征开始
2. 感知机和逻辑回归共用 $z = w^Tx + b$
3. 感知机在犯错或平局后更新
4. sigmoid 把 z 变成概率
5. 二元交叉熵产生简洁的误差 $p - y$

Tutorial 前的五项自检（等于考点清单）

自检项	对应页
顺序地使用更新后的参数	第 10 页（别拿旧 w 算下一个样本）
把标签约定理清楚	第 5、20 页（−1/+1 vs 0/1）
让 $p - y$ 贯穿梯度	第 17 页（**权重梯度和偏置梯度都要写**）
批量更新前先取平均	第 18 页
由 $z = 0$ 画出决策边界	第 15 页（斜截式）

The important task is to connect the ideas, not memorise them as isolated definitions. 检验方法：随便给一组 (w, b, x, y)，能否一口气走完"算 z → 判断/算 p → 出预测 → 算更新"。

第二部分 · Tutorial 活动（Slides 22–25 + 答案）

第 22 页：Tutorial 活动概览

主线：**three core questions connect score → learning → decision boundary**（得分 → 学习 → 决策边界）

题号	考什么	对应课件页
Q1	顺序的感知机更新	第 8–11 页
Q2	对两个逻辑回归梯度贡献取平均	第 17–18 页
Q3	把 $z = 0$ 变成画出来的边界和类别区域	第 15 页
扩展	置信度（confidence extension）	时间允许才做

易错 **Show intermediate values, not only final answers.**（要写出中间值，不能只给最终答案。）课件出现了两次，说明评分看过程。
Q1 要列：每个样本的 s、$y \cdot s$、是否更新、更新后的 w 和 b
Q2 要列：每个样本的 z、p、$p-y$、$(p-y)x$，然后才是平均值
Q3 要列：斜率 $-w_1/w_2$、截距 $-b/w_2$、用测试点验证哪边是类别 1

流程：先独立做 → 小组比较方法 → 讲解后修订答案。

第 23 页：Question 1 的四步工作流——追踪状态，然后更新

The parameters after example i become the starting parameters for example i + 1. 课件说 **Q1 is about state**——考的是参数能不能正确传递。

步骤	做什么	具体写什么
① Compute score	算得分	$s_i = w^T x_i + b$，然后 $\hat{y}_i = \text{sign}(s_i)$
② Check signed score	检查带符号得分	算 $y_i s_i$；为正 → 不更新
③ Update if needed	需要则更新	若 $y_i s_i \leqslant 0$：$w \leftarrow w + \eta y_i x_i$，$b \leftarrow b + \eta y_i$
④ Carry forward	带着走	明确写出新 w 和 b，立即用于下一个样本，并解释得分变化

建议的答题表格：样本 i | 当前 w | 当前 b | s_i | y_i | $y_i s_i$ | 更新? | 新 w | 新 b

重点 **快速自检**：更新完之后把新参数代回去**重算这个样本的得分**，看它是不是朝正确方向移动了。如果反而往错误方向去了，一定是把更新的符号写反了（写成 $w - \eta yx$）。**一次检查就能抓出最常见的符号错误。**
标注中间量的理由：*keep intermediate quantities labelled so that an arithmetic or convention error can be located and corrected*——错了能定位，也拿得到过程分。

第 24 页：Question 2 的四步——让误差穿过链式法则

步骤	做什么	备注
① $z_i \to p_i$	算得分再套 sigmoid	$z = w^T x + b$，$p = \sigma(z)$
② $p_i - y_i$	记录带符号概率误差	**同一个误差同时驱动两个梯度**
③ $\times x_i$	形成权重和偏置的贡献	$\partial z/\partial w = x_i$，$\partial z/\partial b = 1$
④ average → update	先平均，再更新	$w \leftarrow w - \eta \cdot \partial J/\partial w$，$b \leftarrow b - \eta \cdot \partial J/\partial b$

易错 **三个最容易失分的地方**
① **更新是"减"不是"加"**：$w \leftarrow w - \eta\ \partial J/\partial w$。与感知机的 $w \leftarrow w + \eta yx$ 形式相反（感知机的 y 已带符号方向；这里 $p-y$ 是梯度，指向损失上升方向，所以要减）。
② **两个样本都用"相同的起始参数"**——这与 Q1 完全相反：
- Q1（感知机）：**顺序**，样本 2 用样本 1 更新后的参数，每个样本后立刻更新
- Q2（批量逻辑回归）：**并行**，两个样本都用同一组初始参数，取平均后才更新一次

这是两道题之间最大的陷阱。
③ **权重和偏置分别平均**：$\partial J/\partial w = \frac{1}{2}[(p_1-y_1)x_1 + (p_2-y_2)x_2]$；$\partial J/\partial b = \frac{1}{2}[(p_1-y_1) + (p_2-y_2)]$

Keep sufficient decimal precision until the final line. 中间保留足够小数位，最后一行再四舍五入——否则误差会被 ×x、取平均、×η 一路放大。

第 25 页：Question 3——把方程变成图

阶段	要写什么	快速自检
Boundary 边界	令 $z = 0$	有点满足它吗?
Rearrange 重排	化成 $x_2 = mx_1 + c$	**检查两个符号**
Classify 分类	计算 z	看符号
Confidence 置信度	计算 $\sigma(z)$	看离 0.5 有多远

1. 令 $z = 0$（因 $p = 0.5 \Leftrightarrow z = 0$）
2. 重排成斜截式：$\mathbf{m = -w_1/w_2}$，$\mathbf{c = -b/w_2}$。"check both signs"指两个系数前都有负号，最易出错（另需 $w_2 \neq 0$）
3. **选两个点，验证它们都满足 $z = 0$**。画直线只需两点，这一步能立刻抓出符号错误
4. **先画线，再分类**（plot the line before classifying）——顺序不能反
5. 用 z 的符号给两侧命名：$z > 0$ 一侧是类别 1，$z < 0$ 一侧是类别 0

置信度：比较每个概率**离 0.5 的距离**。越远越有把握。所以 $p = 0.9$ 与 $p = 0.1$ **置信度相同**（都离 0.5 有 0.4），只是方向相反；$p = 0.55$ 则很没把握。等价做法是比 $|z|$（σ 单调），但记得 $|z|$ 不是几何距离。

Tutorial Question 1 完整解答

题目：$w = [1, -1]$，$b = 0$，$\eta = 1$。样本 $(x_1,y_1) = ([2,3], +1)$，$(x_2,y_2) = ([-1,-2], -1)$

样本	当前参数	s	$\hat{y}$	$y \cdot s$	更新?	新参数
1	w=[1,−1], b=0	(1)(2)+(−1)(3)+0 = **−1**	−1	(+1)(−1) = **−1 ⩽ 0**	**是**	w = [1,−1]+[2,3] = **[3,2]** b = 0+1 = **1**
2	w=[3,2], b=1	(3)(−1)+(2)(−2)+1 = **−6**	−1	(−1)(−6) = **+6 > 0**	否	不变

最终答案：$\mathbf{w = [3, 2]}$，$\mathbf{b = 1}$

> 易错 样本 2 的 **$s_2 = -6$ 是负数，但样本也是负类，两负相乘得正，所以分对了**。若误用初始参数 $w = [1,-1]$ 会算出 $s_2 = 1$、$y_2 s_2 = -1 \leqslant 0$，得出"要更新"的错误结论，整道题就错了。

题目要求的那句解释：用新参数重算样本 1，$s = (3)(2)+(2)(3)+1 = \mathbf{13}$。这次更新把被分错的正类样本的得分**从 −1 提高到了 13**，把它移到了边界的正确一侧。

Tutorial Question 2 完整解答

题目：单特征（w 是标量）。$z_i = wx_i + b$，$p_i = \sigma(z_i)$。$(x_1,y_1) = (0.5, 1)$，$(x_2,y_2) = (-0.3, 0)$，$w = 1$，$b = 0$，$\eta = 0.1$

$$\partial J/\partial w = \tfrac{1}{2}\,\Sigma(p_i - y_i)x_i \qquad \partial J/\partial b = \tfrac{1}{2}\,\Sigma(p_i - y_i)$$

第 1 步：逐样本算四个量（两个样本都用同一组初始参数 $w = 1, b = 0$）

i	z_i	p_i	$p_i - y_i$	$(p_i - y_i)x_i$
1	0.5	0.622459	−0.377541（概率低于目标，要抬高）	−0.188770
2	−0.3	0.425557	+0.425557（概率高于目标，要压低）	−0.127667

注意：两个误差**符号相反**，但两个 $(p-y)x$ **都是负的**——因为 x_2 本身是负数，乘上 x 之后符号会翻转。

第 2 步：求平均梯度并更新

- $\partial J/\partial w = \tfrac{1}{2}(-0.188770 - 0.127667) = \tfrac{1}{2}(-0.316437) \approx \mathbf{-0.158219}$
- $\partial J/\partial b = \tfrac{1}{2}(-0.377541 + 0.425557) = \tfrac{1}{2}(0.048016) \approx \mathbf{0.024008}$（**大量抵消**！两个样本想把 b 往相反方向拉）
- $w \leftarrow 1 - 0.1(-0.158219) = \approx \mathbf{1.015822}$（梯度为负 → 参数变大）
- $b \leftarrow 0 - 0.1(0.024008) = \approx \mathbf{-0.002401}$（梯度为正 → 参数变小）

第 3 步：解释题

- **为什么 p−y 是"带符号的概率误差"**：预测概率高于目标时为正，低于目标时为负——符号告诉你方向，大小告诉你调多少
- **为什么沿负梯度走一小步能降低损失**：梯度指向损失上升最快的方向，负梯度方向是**局部下坡**。只要步长足够小就不会冲过头。本题平均损失从 **0.514216 降到 0.511660**。"**sufficiently small**"这个限定很关键——步长太大可能跨过谷底反而爬升

Tutorial Question 3 完整解答

题目：$z = 2x_1 - x_2 - 3$，$P(y=1|x) = \sigma(z)$，$p \geqslant 0.5$ 时预测类别 1。（即 $w_1 = 2$，$w_2 = -1$，$b = -3$）

① 为什么边界是 $z = 0$：因为 $\sigma(0) = 0.5$ 且 sigmoid 单调递增，阈值 $p \geqslant 0.5$ 恰好对应 $z \geqslant 0$。

重排：$2x_1 - x_2 - 3 = 0 \Longleftrightarrow \mathbf{x_2 = 2x_1 - 3}$。斜率 $\mathbf{m = 2}$（$= -w_1/w_2 = -2/(-1)$），纵截距 $\mathbf{c = -3}$（$= -b/w_2 = -(-3)/(-1)$）。

> 易错 本题因 $w_2 = -1$，两个负号被抵消，斜率和截距**看起来就像直接从原式移项**。若 w_2 是正数符号就会翻转——别养成"直接搬系数"的习惯。

② 两个方便的点：$x_1 = 0 \to x_2 = -3$，得 $(0, -3)$；$x_1 = 2 \to x_2 = 1$，得 $(2, 1)$。验算：$2(2) - 1 - 3 = 0$ ✓

③ 三个点的得分与类别

点	计算	z	类别
$A = [2, 2]$	$2(2) - 2 - 3$	**−1**	0
$B = [1, 5]$	$2(1) - 5 - 3$	**−6**	0
$C = [3, 1]$	$2(3) - 1 - 3$	**+2**	1

④ 哪一侧是类别 1：$z > 0 \Longleftrightarrow 2x_1 - x_2 - 3 > 0 \Longleftrightarrow \mathbf{x_2 < 2x_1 - 3}$，**即直线的下方**。

> 易错 很多人以为类别 1 在上方。**这里是下方**，因为 **$w_2 = -1$ 是负的**：x_2 越大 z 越小。最稳妥的做法是拿一个点验证：$C = [3,1]$ 的 $z = +2 > 0$，而它在图上位于线的下方 → 下方就是类别 1 区域。**一个点定乾坤。**

扩展题：置信度

点	z	$P(y=1)$	预测	离 0.5 的距离
A	−1	≈ 0.269	0	0.231
B	−6	≈ 0.00247	0	**0.498（最远）**
C	+2	≈ 0.881	1	0.381

B 最有把握，其预测类别 0 的置信度是 $1 - 0.00247 \approx$ **99.75%**。注意 A 和 B **类别相同但把握天差地别**——感知机对两者会给出完全相同的输出，逻辑回归却能告诉你 B 可靠得多。**这就是概率的价值。**

第三部分 · Practical 实践活动（Slides 26–28 + Notebook）

第 26 页：Practical 概览

28 × 28 images become 784 numerical features——一张 28×28 像素的图片摊平（flatten）成 784 维向量，每个特征是一个像素的灰度值。这是第 5 页"分类从表示开始"的真实版本：**维度从 2 变成 784，但 $z = w^T x + b$ 一个字都没变**。

题号	内容	关键点
Q1	检查特征、标签和**类别平衡**	先看数据，别急着建模
Q2	训练并诊断感知机做 **5 vs 非 5**	二分类
Q3	实现 **one-vs-rest**，区分 2、4、6 和其他	多分类
Q4	比较**硬得分**与**逻辑回归概率**	呼应第 19 页
收尾	误差分析 + 简短反思	

类别平衡（class balance）："5 vs 非 5"中 5 只占约 10%，严重不平衡。一个永远预测"非 5"的模型也能拿 90% 准确率但毫无用处。

one-vs-rest（OvR）：为每个类别训练一个二分类器，预测时取得分最高的那个。这是把二分类器扩展到多分类的标准做法。

易错 **justify conclusions from evidence rather than from a single score**——结论要有证据支撑，不能只看一个准确率。
How will you avoid using test data to make development decisions?——调参、选模型、改特征都必须只用训练集/验证集。一旦用测试集表现来挑模型，测试集就被污染，报出的性能会偏乐观（数据泄漏在模型选择环节的体现）。

第 27 页：从像素到决策

阶段	做什么	细节
① Represent the digits	表示数字	每张图变成 784 个像素特征
② Binary perceptron	二分类感知机	5 vs 非 5；检查预测结果
③ One-vs-rest	一对其余	分类 2、4、6 和其他（共 4 类）
④ Add probabilities	加入概率	逻辑回归；解释置信度

四步是**递进**的：先建立表示 → 用最简单的模型做最简单的任务 → 扩展到多分类 → 再升级到概率输出。每步只增加一个新东西，方便定位问题。

重点 **Classification reuses the linear score from regression but interprets it as a class decision or probability.**（分类复用了回归的线性得分，只是把它解释成类别判定或概率。）
Perceptron and logistic regression differ mainly in their output transformation, loss, and update signal.（三个词：输出变换 sign vs σ；损失 $\max(0,-ys)$ vs 交叉熵；更新信号 $y \cdot s$ vs $p-y$。）

Study check：说出输入、输出和一个假设或局限

- **输入**：784 维像素向量；**输出**：类别标签（或概率）
- **局限①**：模型是**线性的**——只能画超平面，手写数字未必线性可分
- **局限②**：像素特征**没有空间感**——摊平后模型不知道哪两个像素在图上相邻，平移一点图像就可能认不出
- **局限③**：感知机**不给概率**，无法表达置信度
- **假设**：训练集和测试集来自**同一分布**

第 28 页：如何完成 notebook

步骤	做什么	细节
① Run in order	按顺序运行	从顶部开始，每个 cell 运行一次
② Complete the code	补全代码	完成 TODO 和 one-vs-rest 编程挑战
③ Diagnose results	诊断结果	**比较模型、检查错误、给出解释**
④ Reflect and extend	反思与扩展	总结所学；用官方文档；**扩展题放最后**

为什么要按顺序运行：Jupyter 的 cell **共享同一个内存状态**。跳着运行会用到未定义或过时的变量，报错莫名其妙且**结果不可复现**。遇到怪错就 Restart Kernel + Run All。

重点 **The practical emphasises correct experimental reasoning as well as code.**（代码跑通只是及格线。）还要能：比较模型（感知机 vs 逻辑回归、硬得分 vs 概率）、检查错误（哪些数字被认错？错的图长什么样？）、结合线性边界/类别不平衡/像素丢失空间信息来解释原因。

Notebook Setup（cell 6）

```
RANDOM_STATE = 42                      # 固定随机种子，保证结果可复现
plot_digit: image_data.reshape(28, 28) # 784 维一维向量还原成二维图像
```

```
dataset = fetch_openml("mnist_784", version=1, as_frame=False, cache=True)
```

三行关键预处理

代码	作用
`X = np.asarray(X_raw, dtype=np.float32) / 255.0`	**特征缩放**：像素 0–255 归一化到 [0,1]。若不缩放，x 高达 255 会让 w ← w + ηyx 一次推得太远，训练极不稳定。注意这**不是** StandardScaler 的标准化，而是固定除法——0–255 的范围天然已知，**不存在数据泄漏问题**
`y = np.asarray(y_raw).astype(str)`	**标签是字符串**！后面必须写 `y_train == "5"`（带引号）。写成 `== 5` 会得到全 False，不报错但结果全错——这种错最难找
`X_train, X_test = X[:60000], X[60000:]`	**用 MNIST 官方标准划分**而不是 train_test_split：所有人结果可比较，且 MNIST 已打乱过

输出：训练集 (60000, 784)，测试集 (10000, 784)。

Practical Question 1 — 从图像到特征

```
fig, axes = plt.subplots(2, 10, figsize=(10, 2.6))   # 2 行 10 列：每个数字一列，两个样本
for digit in range(10):
    digit_indices = np.flatnonzero(y_train == str(digit))
    for row in range(2):
        plot_digit(X_train[digit_indices[row]], axes[row, digit])
        if row == 0: axes[row, digit].set_title(str(digit))
```

- `y_train == str(digit)` → 布尔数组（str() 是因为标签是字符串）
- `np.flatnonzero(...)` → 把布尔数组变成**"True 所在位置的下标数组"**
- `if row == 0` → 只在第一行加标题，避免重复

参考答案
① **同一数字的变化**：位置（position）、笔画粗细（stroke thickness）、弯曲程度（curvature）、倾斜（slant）
② **可能混淆的数字对**：**4 和 9**（形状重叠时容易混）
③ **784 个特征之一代表什么**：**某个固定位置上单个像素的归一化亮度值**

重点 **"固定位置"**三个字是核心：第 k 个特征永远对应图上同一位置的像素（第 5 页"特征顺序必须一致"）。这直接导致局限——**图像平移几个像素，所有特征值都变了，但模型不知道这只是同一个数字挪了位置**。线性模型没有空间感知能力。

Practical Question 2 — 二分类感知机"这是不是 5？"

```
y_train_5 = (y_train == "5");  y_test_5 = (y_test == "5")   # 重新定义目标 → 二分类
binary_perceptron = Perceptron(random_state=RANDOM_STATE)
binary_perceptron.fit(X_train, y_train_5)
binary_pred  = binary_perceptron.predict(X_test)             # 硬标签，相当于 sign(z)
binary_score = binary_perceptron.decision_function(X_test)   # 原始线性得分 z
print("Majority baseline:", max(y_test_5.mean(), 1 - y_test_5.mean()))
```

目标定义（target definition）决定了分类问题是什么——这两行不改变任何特征，只改变标签。

多数类基线：`y_test_5.mean()` 是 5 的占比 ≈ 0.0892，取它与 1−它 的较大者 = **0.9108**，即"永远预测不是 5"能拿到的准确率。

指标	值
多数类基线	**0.9108**
感知机测试准确率	**0.9651**（超过了基线）
前五个得分	−213.39, −56.32, −49.00, −223.95, −226.69

参考答案要点
② **为什么准确率可能有误导性**：测试集里**只有 8.92% 是 5，高的整体准确率可以和"漏掉很多 5"同时存在**。算一下：模型答错 3.49%（≈349 张），若这些错误全是把 5 误判成非 5，等于漏掉 349/892 ≈ **39% 的 5**。（Q4 会加入 precision 和 recall 来暴露这类错误。）
③ **得分的符号**：正得分表示图像落在"是 5"一侧，负得分落在"不是 5"一侧；**绝对值越大表示离拟合出的边界越远（in score units 以得分为单位）**，但**这不是概率**——概率不可能是 −213。

Practical Question 3 — 四类 one-vs-rest

```
selected_digits = ["2", "4", "6"]
y_train_4 = np.where(np.isin(y_train, selected_digits), y_train, "others")
multi = Perceptron(random_state=RANDOM_STATE);  multi.fit(X_train, y_train_4)
ConfusionMatrixDisplay.from_predictions(y_test_4, multi_pred, normalize="true", ...)
```

- `np.isin(...)` → 布尔数组：该样本是不是 2、4 或 6

- `np.where(条件, A, B)` → 条件为真取 A，为假取 B（逐元素三元运算符）。**是 2/4/6 保留原标签，否则改成 "others"**，形成 4 类问题
- **建模代码一个字都没改**——sklearn 检测到标签有 4 个值，**自动启用 one-vs-rest**，内部训练 4 个二分类器
- `normalize="true"` → 按**真实类别按行归一化**，每行和为 1，**对角线上的数字就是每个类别的 recall**（按列归一化则是 precision）

指标	值	类别	recall
多数类基线（others 占比）	**0.7028**	2	**≈ 0.750（最低）**
四类感知机测试准确率	**0.9311**	4	≈ 0.925
		6	≈ 0.932
		others	≈ 0.958

参考答案的四点
① **one-vs-rest 怎么产生四类预测：为每个类别学一个得分，然后选得分最大的那个类别**（argmax）。注意用的是得分**大小**不是符号——即使 4 个得分全负，也选最不负的那个。
② **哪一类 recall 最低：数字 2（0.750）**，且**约 22.2% 的真实 2 被预测成 others**。原因：others 里的 7、3、8 与 2 笔画重叠，且 others 训练样本远多于其他三类。
③ **为什么 others 要小心解读：others 很宽泛，包含 7 种视觉上完全不同的数字，高 recall 并不意味着它是一个连贯的视觉类别**。模型不是"学会了 others 长什么样"，而是"把不像 2/4/6 的都往这儿扔"。
④ 0.9311 > 0.7028，超过基线。

Practical Question 4 Part A — 从硬判定到概率

```
logistic_train_size = 15_000        # 只用 15000 张（感知机用了全部 60000），测试集完全相同
logistic_model = LogisticRegression(solver="liblinear", max_iter=300, random_state=RANDOM_STATE)
logistic_probability = logistic_model.predict_proba(X_test)[:, 1]   # 第 1 列 = P(是5)
logistic_prediction  = logistic_probability >= 0.5                  # 手动阈值化
perceptron_probability = 1 / (1 + np.exp(-np.clip(binary_score, -50, 50)))  # 硬套 sigmoid
```

`np.clip(..., -50, 50)` 是**数值稳定处理**：得分有 −226 这么大，exp 会溢出，先截断到 ±50（σ(±50) 已到机器精度的 0 或 1）。

模型	accuracy	precision	recall	log loss
感知机	0.9651	**0.9010**	0.6839	**0.8440**
逻辑回归	**0.9744**	0.8888	**0.8150**	**0.0919**

precision＝预测为 5 的里面真是 5 的比例（衡量误报）；**recall**＝真实的 5 里面被找出来的比例（衡量漏检）。

参考答案解读
① **为什么只看准确率不够**：逻辑回归**找到了多得多的真 5**（recall 0.8150 vs 0.6839），感知机 precision 略高（0.9010 vs 0.8888）。感知机 recall 只有 0.6839 意味着**漏掉约 32% 的 5**，但准确率 0.9651 看起来很漂亮——**准确率把这个问题完全藏起来了**。这是典型的 precision-recall 权衡。
② **log loss 差近 10 倍**：log loss 就是二元交叉熵，衡量的是**概率报得准不准**而不只是类别猜得对不对。感知机的"伪概率"被大量"自信地错了"的样本狠狠惩罚。
③ **sigmoid(score) 与 predict_proba 的最大差值是 0.0**（到数值精度）——证实**逻辑回归的 predict_proba 内部就是 σ(decision_function)**，即课件第 13 页的 p = σ(z)。

易错 ④ **为什么套了 sigmoid 的感知机得分仍然不是好概率**：会得到 0 到 1 之间的值，**但这不能使它们成为校准良好的概率，因为感知机不是用逻辑损失训练的**。
- **形式上像概率 ≠ 实际上是概率**。σ 能把任何实数压到 (0,1)，但压出来的数没有意义。
- 根本原因：**训练目标决定了输出的含义**。逻辑回归的参数是为了最小化交叉熵才调成那样；感知机只为"分对边"，**得分的尺度是任意的**（w、b 同乘 10 边界不变但得分全变 10 倍，套上 sigmoid 后概率完全不同）。

直方图：画真 5 与非 5 两组的预测概率分布 + 0.5 阈值虚线。理想是两堆分别挤在 1 和 0 附近，**重叠区域就是模型没把握的样本**。

Practical Question 4 Part B — 手动执行一次梯度更新

```
X_toy = [[-2,-1], [-1,-1.5], [1,0.5], [2,1.5]];  y_toy = [0, 0, 1, 1]
w = [0, 0];  b = 0;  learning_rate = 0.5
probability_before = sigmoid_stable(X_toy @ w + b)      # 矩阵乘向量，一次算出 4 个 z
gradient_w = X_toy.T @ (probability_before - y_toy) / len(y_toy)
gradient_b = np.mean(probability_before - y_toy)
w = w - learning_rate * gradient_w;   b = b - learning_rate * gradient_b
```

更新前：w、b 全零 → 所有 z = 0 → 所有 **p = 0.5** → 损失 = −log(0.5) = **0.6931**。

重点 **0.6931 = ln 2** 是"完全没头绪、每个都猜 50%"时的交叉熵，是二分类的**随机猜测基准**。任何有用的模型 log loss 都该明显低于它。

梯度计算（第 17 页公式的向量化写法）：`probability_before - y_toy` 就是 $p - y = [0.5, 0.5, -0.5, -0.5]$；`X_toy.T @ (...)` 等价于 $\Sigma(p_i - y_i)x_i$；`/ len(y_toy)` 就是取平均。

手算验证第一维：$[(-2)(0.5) + (-1)(0.5) + (1)(-0.5) + (2)(-0.5)]/4 = -3/4 =$ **-0.75** ✓

量	值
Gradient for w	**[−0.75, −0.5625]**
Gradient for b	**0.0**
更新后 w	$w_1 = 0 - 0.5(-0.75) =$ **0.375**；$w_2 =$ **0.28125**
更新后 b	**0**（梯度为 0，不变）
BCE before → after	**0.6931 → 0.3538**（几乎腰斩）

> **为什么偏置梯度正好是 0？** $\partial J/\partial b = \text{mean}(p - y) = (0.5 + 0.5 - 0.5 - 0.5)/4 = 0$。**两个正类和两个负类数量相等**，初始误差完全对称地抵消。（呼应 tutorial Q2 里偏置梯度几乎被抵消的现象，只不过这里是完全抵消。）

画边界：`x2_boundary = -(w[0]*x1_boundary + b) / w[1]` 就是 $w_1x_1 + w_2x_2 + b = 0$ 解出 x_2——**和 tutorial Q3 的斜截式完全一致**。代入数字：斜率 = −0.375/0.28125 = **−4/3**，截距 = 0。

参考答案措辞：减去梯度**在所选的学习率下**把参数推向了降低损失的方向；图上画的是更新后的边界 w[0]×x1 + w[1]×x2 + b = 0，**位于两侧的观测点会得到相反的阈值化预测**。

Practical 最终反思（Final reflection）

问题：对真实的文档识别系统，除了测试准确率之外还有哪些数据偏移或错误代价需要考虑?

> **参考答案**
> ① **数据偏移**：书写风格、扫描质量、语言、设备条件都可能与 MNIST 不同。（MNIST 是 1990 年代美国人写的、经过居中和尺寸归一化的数字；真实场景可能是欧洲式的 7 带横杠、手机拍照的模糊倾斜、彩色背景等。）
> ② **代价不对等的错误**：**读错账号**可能比"**拒绝一张不清晰的图、转人工审核**"严重得多。前者可能把钱打给错误的人（灾难性），后者只是多花人工（可承受）。

> 易错 **③ 为什么整体准确率反映不了这两点（两个原因要分开说）**
> - **对数据偏移**：测试集和训练集来自**同一分布**，0.9744 只衡量了"**分布内**"表现，对分布外的新数据没有预测力。
> - **对错误代价**：准确率**把所有错误一视同仁**，没有代价加权的概念。

延伸：既然逻辑回归给出概率，就可以设计**拒答（reject option）**机制——p 落在 0.4–0.6 之间时不下判断，转人工。**这正是概率的实用价值，感知机做不到。**

可选扩展题

1. 换个目标数字（如"3 vs 非 3"）重做感知机/逻辑回归对比，验证结论是否稳健
2. **改变玩具数据的学习率，找出哪些值会降低损失、哪些会增加损失**——直接检验 tutorial Q2 那句"**sufficiently small step**"。η = 0.5 时损失从 0.6931 降到 0.3538；若取 50，参数被推得极远，**损失反而暴涨**。动手试一次就永远记住为什么步长要小

Practical Summary — 六条总结

1. **图像变成数值特征向量**（Q1）
2. **目标的定义决定了分类问题是什么**（Q2、Q3——同样的特征，改标签就换了问题）
3. **感知机和逻辑回归形成的都是线性决策边界**
4. **逻辑回归把得分、sigmoid 概率和二元交叉熵连成一条链**
5. **one-vs-rest 把二分类扩展到多分类**
6. **基线、分类别指标和错误检查比单看准确率揭示得更多**

第 2 条和第 6 条是 practical 相对课件**新增**的内容，值得单独记。

第四部分 · 阅读材料《Logistic Regression: A Step Beyond the Perceptron》

这份 Canvas 阅读材料是课件第 13–18 页的文字版补充，多给了几个课件上没有的细节（尤其是 sigmoid 导数和 logit）。

1. 为什么要从感知机走向逻辑回归

感知机作为二分类的起点是好的，但有两个局限：**数据不线性可分时**，以及**我们想知道模型有多确信**时。逻辑回归提供了更灵活的做法。

	感知机	逻辑回归
输出	**离散**：+1（happy）或 −1（sad），中间什么都没有	**连续**：0 到 1 之间，可解释为概率
能回答的问题	"这句话是 happy"	"这句话有 88% 的可能是 happy"

材料举的例子：句子 "I love sunny days" 可能被判为 happy，**但有多 happy？是 90% 确定，还是勉强算数？** 感知机答不了这个问题。

2. 阶跃函数 vs. Sigmoid 函数

感知机用**阶跃函数**做决策：$z = w^T x + b$ 为正输出 +1，为负输出 −1。这造成一个**硬决策边界**，而且——

> **the step function is not smooth and therefore does not provide a useful gradient for gradual optimization**
> 阶跃函数不平滑，因此**提供不了对逐步优化有用的梯度**。这是换成 sigmoid 的根本理由。

$$\sigma(z) = 1 / (1 + e^{-z})$$

其中 $z = w^T x + b$ 就是感知机里那个加权和。sigmoid 的输出解释为**输入属于正类（happy）的概率**：接近 1 表示"很可能是 happy"，接近 0 表示"很可能是 sad"。

3. 四个步骤（Step-by-Step Explanation）

Step 1：逻辑模型

先算线性组合（和感知机一样）：**$z = \mathbf{w}^T\mathbf{x} + \mathbf{b}$**，其中 x 是特征向量（如 happy/sad 词数）、w 是权重向量、b 是偏置。然后**不直接用 z 分类**，而是把 z 送进 sigmoid 得到 0 到 1 的概率：

$$P(\text{happy}) = \sigma(z) = 1 / (1 + e^{-z})$$

Step 2：解释输出

写作 **$p = P(y = 1 \mid x) = \sigma(z)$**。用常规阈值 0.5：**$p \geqslant 0.5$ 预测类别 1，$p < 0.5$ 预测类别 0**。例：模型输出 P(y=1|x) = 0.88，就是给正类分配了 0.88 的估计概率，在 0.5 阈值下预测类别 1。

> 易错 **Label convention**：这份笔记里**感知机用 {−1, +1}，逻辑回归用 {0, 1}。比较公式时，两套约定之间的换算很重要。**

The Decision Boundary（决策边界）

因为 σ(0) = 0.5，0.5 概率的边界正好出现在线性得分为零的地方：

$$P(y = 1 \mid x) = 0.5 \iff z = 0 \iff w^T x + b = 0$$

两个特征时 $w_1x_1 + w_2x_2 + b = 0$，当 $w_2 \neq 0$ 重排得 **$x_2 = -(w_1/w_2)x_1 - b/w_2$**。材料给的例子正是 tutorial Q3：**$2x_1 - x_2 - 3 = 0$ 变成 $x_2 = 2x_1 - 3$，两个方便的点是 (0, −3) 和 (2, 1)**。

> **The sigmoid changes scores into probabilities, but the default logistic-regression boundary is still linear.**

Step 3：损失函数

目标是找到**使正确分类的似然（likelihood）最大**的 w 和 b。不像感知机那样用简单的误差项更新权重，我们最小化一个特殊的误差函数——**logistic loss**，也叫**二元交叉熵**：

$$L(y_{true}, P(\text{happy})) = -(\, y_{true} \cdot \log(P(\text{happy})) + (1 - y_{true}) \cdot \log(1 - P(\text{happy}))\,)$$

y_{true} 是真实标签（happy 为 1，sad 为 0），P(happy) 是预测概率。**模型对错误越自信，这个损失惩罚越重**：若真实是 happy（y = 1）但模型预测 P(happy) = 0.1，损失会很大，迫使模型更新权重来减小这个误差。

From Probability Error to Gradients（新增细节）

材料在这里给了课件上**没有明说**的一步：

$$\sigma'(z_i) = p_i(1 - p_i) \qquad \text{（sigmoid 的导数）}$$

把这个导数与二元交叉熵通过**链式法则**结合，得到那个特别简洁的结果：

$$\partial L_i / \partial z_i = p_i - y_i$$

$p_i - y_i$ 是模型的 **signed probability error**：**为负则该样本的得分应该增加，为正则得分应该减少**。

Step 4：用梯度下降更新权重

$$\partial L_i / \partial w = (p_i - y_i)x_i \qquad \partial L_i / \partial b = p_i - y_i$$

对 n 个样本的数据集，平均梯度是：

$$\nabla_w J = (1/n)\, \Sigma_{i=1}^{n}\, (p_i - y_i)x_i \qquad \partial J / \partial b = (1/n)\, \Sigma_{i=1}^{n}\, (p_i - y_i)$$

梯度下降沿**梯度的反方向**更新参数：

$$w \leftarrow w - \eta \nabla_w J \qquad b \leftarrow b - \eta \cdot \partial J / \partial b$$

用全部样本是批量更新，一次用一个样本是随机更新。无论哪种，**特征向量决定了哪些权重获得最大的贡献**（因为梯度里乘了 x_i）。

4. Sigmoid vs. 阶跃函数的具体例子

还是 "I love sunny days"。感知机：$w^Tx + b > 0$ 就是 happy，< 0 就是 sad，二元的。逻辑回归则把加权和送进 sigmoid 得到概率：

$$\text{若 } w^Tx + b = 2\text{，则 } \sigma(2) = 1/(1 + e^{-2}) \approx \mathbf{0.88}$$

所以逻辑回归输出 0.88，表示模型给"这句话是 happy"分配了 0.88 的估计概率。**这个概率比单纯的二元预测提供了更丰富的理解。**

第五部分 · 数学推导详解（Derivations）

以下推导按依赖顺序排列，前面的结论会被后面用到。考试未必要求全写，但理解推导能让你在记不清公式时自己推回来。

推导 1：sigmoid 的导数 $\sigma'(z) = \sigma(z)[1 - \sigma(z)]$

令 $\sigma(z) = (1 + e^{-z})^{-1}$。用链式法则对 z 求导：

$$\sigma'(z) = -1 \cdot (1 + e^{-z})^{-2} \cdot d/dz(1 + e^{-z}) = -(1 + e^{-z})^{-2} \cdot (-e^{-z}) = e^{-z} / (1 + e^{-z})^2$$

把它拆成两个因子：

$$\sigma'(z) = [\, 1/(1 + e^{-z}) \,] \cdot [\, e^{-z}/(1 + e^{-z}) \,]$$

第一个括号就是 $\sigma(z)$。第二个括号呢？注意：

$$1 - \sigma(z) = 1 - 1/(1 + e^{-z}) = (1 + e^{-z} - 1)/(1 + e^{-z}) = e^{-z}/(1 + e^{-z})$$

正好是第二个括号。所以：

$$\boldsymbol{\sigma'(z) = \sigma(z)[1 - \sigma(z)] = p(1 - p)}$$

> **三个推论**
> ① **σ' 恒为正**（因为 $0 < p < 1$），所以 sigmoid **严格单调递增**——这正是第 15 页"$p = 0.5 \Leftrightarrow z = 0$"能成立的依据。
> ② **最大值在 $z = 0$ 处**：$p = 0.5$ 时 $\sigma' = 0.25$，是导数的峰值。
> ③ **两端饱和（saturation）**：$|z|$ 很大时 p 接近 0 或 1，$\sigma' \to 0$。这就是"sigmoid 会导致梯度消失"的来源。

推导 2：核心结果 $\partial L/\partial z = p - y$

这是整个 topic 最重要的推导，写法有两种。

方法 A：一般式（先对 p 求导，再链式）

$L = -[\, y \cdot \log p + (1-y) \cdot \log(1-p) \,]$。先对 p 求导（注意 $\log(1-p)$ 求导要乘内层导数 -1）：

$$\partial L/\partial p = -[\, y/p - (1-y)/(1-p) \,]$$

通分，公分母是 $p(1-p)$：

$$\partial L/\partial p = -[\, y(1-p) - (1-y)p \,] / [p(1-p)]$$

展开分子：$y(1-p) - (1-y)p = y - yp - p + yp = \mathbf{y - p}$。所以：

$$\partial L/\partial p = -(y - p)/[p(1-p)] = (p - y) / [p(1-p)]$$

再用推导 1 的 $\partial p/\partial z = p(1-p)$，链式法则：

$$\partial L/\partial z = (\partial L/\partial p) \cdot (\partial p/\partial z) = \mathbf{(p - y)}/[p(1-p)] \cdot \mathbf{p(1-p)} = \mathbf{p - y}$$

> **$p(1-p)$ 被完美约掉了。**这就是为什么 sigmoid 一定要配交叉熵——这两个函数是"天生一对"。

方法 B：分情况验证（考试时更快）

情况	L	$\partial L/\partial p$	$\times\, p(1-p)$	结果
$y = 1$	$-\log p$	$-1/p$	$-(1-p)$	$p - 1 = p - y$ ✓
$y = 0$	$-\log(1-p)$	$+1/(1-p)$	p	$p - 0 = p - y$ ✓

两种情况都得到 $p - y$，所以结论对 $y \in \{0,1\}$ 普遍成立。

推导 3：为什么不能用 MSE 配 sigmoid（反面对照）

假设我们偷懒，用回归的均方误差 $L = \frac{1}{2}(p-y)^2$。那么 $\partial L/\partial p = p - y$，链式后：

$$\partial L/\partial z = (p-y)\cdot \mathbf{p(1-p)}$$

这里 **p(1−p) 没有被约掉**。看一个"自信地错了"的例子：真实 $y = 1$，但模型给出 $p = 0.001$。

损失	$\partial L/\partial z$	后果
交叉熵	$p - y = -0.999$	梯度很大，模型被**猛烈修正**
MSE	$(-0.999)(0.001)(0.999) \approx \mathbf{-0.001}$	梯度几乎为零，模型**纹丝不动**，卡死

> **结论**：交叉熵的 $p - y$ 恰好抵消了 sigmoid 的饱和因子，所以"错得越离谱、修正越猛"；MSE 则会在最需要修正的地方梯度消失。这就是分类必须用交叉熵、不能用 MSE 的数学理由。

推导 4：从 $\partial L/\partial z$ 到 $\partial L/\partial w$ 与 $\partial L/\partial b$

因为 $z = w^T x + b = w_1x_1 + w_2x_2 + \cdots + w_dx_d + b$，逐个分量求偏导：

$$\partial z/\partial w_j = x_j \text{（其余项与 } w_j \text{ 无关，全为 0）} \qquad \partial z/\partial b = 1$$

写成向量形式 $\partial z/\partial w = x$。再用链式法则：

$$\partial L/\partial w = (\partial L/\partial z)\cdot(\partial z/\partial w) = \mathbf{(p-y)\cdot x} \qquad \partial L/\partial b = (\partial L/\partial z)\cdot(\partial z/\partial b) = \mathbf{(p-y)\cdot 1 = p-y}$$

> **两个立刻可得的结论**：
> ① $\mathbf{x_j = 0 \rightarrow \partial L/\partial w_j = 0}$，该权重这一步不动（第 17 页那条性质的来源）。
> ② **偏置的梯度不乘特征**，所以即使特征全为 0，b 依然会更新。可以把 b 看成一个恒等于 1 的虚拟特征 x_0 的权重——这样 $\partial z/\partial b = x_0 = 1$，两个公式其实是同一个。

推导 5：平均损失、批量梯度与矩阵形式

训练目标是平均损失（第 16 页）：$J = (1/n)\sum_{i=1}^{n} L_i$。求导可以逐项进行（导数的线性性）：

$$\nabla_w J = (1/n)\sum (p_i - y_i)x_i \qquad \partial J/\partial b = (1/n)\sum (p_i - y_i)$$

矩阵形式：把所有 x_i 按行堆成 $n \times d$ 的矩阵 X，误差堆成列向量 $(p-y)$，则

$$\nabla_w J = (1/n)\cdot X^T(p-y)$$

这正是 practical Q4B 那行代码 `X_toy.T @ (probability_before - y_toy) / len(y_toy)` 的数学出处。

更新式（沿负梯度）：$w \leftarrow w - \eta\nabla_w J$，$b \leftarrow b - \eta\cdot\partial J/\partial b$。取 n = 全部样本是批量；$n = 1$ 是随机（SGD）；n = 小批就是 mini-batch。**三者是同一个公式的不同 n。**

推导 6：交叉熵是从最大似然（MLE）推出来的

阅读材料说"找到使正确分类的**似然最大**的 w 和 b"，为什么这等价于最小化交叉熵？推导如下。

① 单个样本的概率模型。模型认为 y 服从伯努利分布，$P(y=1|x) = p$，$P(y=0|x) = 1 - p$。这两式可以合写成一个：

$$P(y \mid x) = p^y(1-p)^{1-y}$$

验证：$y = 1$ 时得 $p^1(1-p)^0 = p$ ✓；$y = 0$ 时得 $p^0(1-p)^1 = 1-p$ ✓。

② 整个数据集的似然（假设样本独立，所以连乘）：

$$L(w,b) = \prod_{i=1}^{n} p_i^{y_i}(1-p_i)^{1-y_i}$$

③ 取对数（连乘变连加，且 log 单调不改变最优解位置）：

$$\ell(w,b) = \sum_{i=1}^{n}\left[\, y_i \log p_i + (1-y_i)\log(1-p_i)\,\right]$$

④ 最大化 ℓ 等价于最小化 $-\ell$，再除以 n 取平均：

$$J = -(1/n)\sum\left[\, y_i \log p_i + (1-y_i)\log(1-p_i)\,\right] = \textbf{平均二元交叉熵}$$

结论：最大化似然 ≡ 最小化交叉熵。交叉熵不是随便设计的，它就是伯努利模型的负对数似然。这也解释了为什么交叉熵能产生**校准的**概率——它的目标本来就是让模型给出的概率尽可能贴合数据的真实生成过程。

推导 7：logit（对数几率）——sigmoid 的逆函数

从 $p = 1/(1+e^{-z})$ 反解出 z：

$$1/p = 1 + e^{-z} \to e^{-z} = (1-p)/p \to -z = \ln[(1-p)/p] \to \mathbf{z = \ln[\, p/(1-p)\,]}$$

$p/(1-p)$ 叫做**几率（odds）**，它的对数叫 **log-odds 或 logit**。所以：

线性得分 z 的真正含义是"对数几率"，不是概率。

- $p = 0.5 \to \text{odds} = 1 \to \ln 1 = 0 \to \mathbf{z = 0}$（再次印证边界位置）
- $p = 0.88 \to \text{odds} \approx 7.33 \to z \approx 2$（对应阅读材料 $\sigma(2) \approx 0.88$ 的例子，反过来验证）
- **w_j 的解释**：x_j 每增加 1 个单位，log-odds 增加 w_j，即几率乘以 e^{w_j}。这是逻辑回归系数的标准解读方式。

顺带解释了为什么"逻辑回归"叫回归：**它对 logit 做的是线性回归**。

推导 8：对称性 $\sigma(-z) = 1 - \sigma(z)$

$$\sigma(-z) = 1/(1+e^{z})$$

分子分母同乘 e^{-z}：

$$\sigma(-z) = e^{-z}/(e^{-z}\cdot e^{z} + e^{-z}) = e^{-z}/(1+e^{-z}) = \mathbf{1 - \sigma(z)}$$

（最后一步用了推导 1 中已经算过的 $1-\sigma(z) = e^{-z}/(1+e^{-z})$。）

用途：查表时只需记一半的值。$\sigma(-2) = 1 - 0.881 = 0.119$；$\sigma(-6) = 1 - 0.998 = 0.002$。**几何意义**：sigmoid 曲线关于点 $(0, 0.5)$ 中心对称。

推导 9：感知机更新规则是从它的损失推出来的

感知机损失 $L = \max(0, -y\cdot s)$，其中 $s = w^{T}x + b$。分两段看：

区域	L 的表达式	$\partial L/\partial w$	$\partial L/\partial b$
$y\cdot s > 0$（分对）	$L = 0$	0	0
$y\cdot s < 0$（分错）	$L = -y(w^{T}x + b)$	$\mathbf{-y\cdot x}$	$\mathbf{-y}$

在分错的区域做梯度下降（沿负梯度走）：

$$w \leftarrow w - \eta(-y\cdot x) = \mathbf{w + \eta\cdot y\cdot x} \qquad b \leftarrow b - \eta(-y) = \mathbf{b + \eta\cdot y}$$

感知机的"加号"和逻辑回归的"减号"其实是同一件事：两者都是 $w \leftarrow w - \eta\cdot$(梯度)。只是感知机在分错区域的梯度是 $-yx$（本身带负号），减去它就变成了加。**不是两套规则，是同一套规则在两个不同损失上的结果。**

关于 $y\cdot s = 0$ 这一点：$\max(0, \cdot)$ 在拐点处**不可导**（左导数 $-y\cdot x$，右导数 0，不相等），所以这里只能取**次梯度（subgradient）**。次梯度可以取区间内任意值，本课程约定取"更新"那一侧，即 $y\cdot s \leqslant 0$ 就更新。**这就是那条 $\leqslant$ 规则的数学来源。**

推导 10：感知机更新后带符号得分必然增加

设更新前得分 $s = w^{T}x + b$，更新后 $w' = w + \eta yx$，$b' = b + \eta y$。计算新得分：

$$s' = w'^{T}x + b' = (w+\eta yx)^{T}x + b + \eta y = w^{T}x + \eta y\cdot x^{T}x + b + \eta y = s + \eta y(\|x\|^{2} + 1)$$

两边同乘 y，并利用 $\mathbf{y^{2} = 1}$（因为 $y = \pm 1$）：

$$y\cdot s' = y\cdot s + \eta y^{2}(\|x\|^{2} + 1) = \mathbf{y\cdot s + \eta(\|x\|^{2} + 1)}$$

结论：因为 $\eta > 0$ 且 $\|x\|^{2} + 1 > 0$，所以 **$y\cdot s'$ 严格大于 $y\cdot s$**，增量恰好是 $\eta(\|x\|^{2} + 1)$。

- 这证明了每次更新**一定**把犯错的样本往正确方向推，方向永远不会错。
- 但**不保证一次就翻过边界**——要 $y\cdot s' > 0$ 需要增量足够大，取决于 η 和 $\|x\|$。
- **验算第 11 页的例子**：$\eta = 0.5$，$x = [2,0]$，$\|x\|^{2} = 4$，增量 $= 0.5\times(4+1) = 2.5$。原 $y\cdot s = -1$，新 $y\cdot s = -1 + 2.5 = 1.5$ ✓ 与直接算出的 $s' = 1.5$（$y = +1$）完全一致。
- **验算 tutorial Q1**：$\eta = 1$，$x = [2,3]$，$\|x\|^{2} = 4 + 9 = 13$，增量 $= 1\times(13+1) = 14$。原 $y\cdot s = -1$，新 $y\cdot s = -1 + 14 = 13$ ✓ 与重算得到的 13 一致。

推导 11：点到决策边界的垂直距离 $= |z| / \|w\|$

① **w 是边界的法向量。**设 x_a、x_b 是边界上任意两点，则 $w^Tx_a + b = 0$ 且 $w^Tx_b + b = 0$。相减得

$$w^T(x_a - x_b) = 0$$

即 w 与边界上任意方向向量都垂直，所以 w 是边界的法向量。

② **投影求距离。**设 x_0 是任意一点，x_p 是边界上一点。x_0 到边界的垂直距离，就是向量 $(x_0 - x_p)$ 在**单位法向量** $w/\|w\|$ 上的投影长度：

$$d = |\,(w/\|w\|)^T(x_0 - x_p)\,| = |\,w^Tx_0 - w^Tx_p\,| \,/\, \|w\|$$

③ **代入边界条件。**因为 x_p 在边界上，$w^Tx_p = -b$，所以

$$d = |\,w^Tx_0 + b\,| \,/\, \|w\| = \mathbf{|z| \,/\, \|w\|}$$

推导完毕。这就是第 7 页那个公式的来源。

推导 12：为什么得分的尺度是任意的（缩放不变性）

把参数同时乘以任意常数 $c > 0$，记 $w' = cw$、$b' = cb$。

量	变化	说明
边界	**不变**	$\{x : cw^Tx + cb = 0\} = \{x : w^Tx + b = 0\}$，同一条线
分类结果	**不变**	$\text{sign}(cz) = \text{sign}(z)$，每个点归属不变
得分 z	**变成 c 倍**	$z' = cz$
几何距离	**不变**	$\|cz\|/\|cw\| = c|z|/(c\|w\|) = |z|/\|w\|$
σ(z)	**完全改变**	$\sigma(z) \neq \sigma(cz)$。例：$z = 2$ 时 $\sigma = 0.88$；$c = 10$ 后 $z' = 20$，$\sigma \approx 0.9999999979$

易错 **这就是"感知机的得分套上 sigmoid 也不是好概率"的严格证明。**感知机的训练只关心分类是否正确，而分类结果对缩放不变，所以**它对 w 的尺度没有任何约束**——同一个感知机可以有无穷多组等价参数，但它们给出的"伪概率"从 0.88 到 0.9999999979 任意变化。
逻辑回归为什么没有这个问题？因为**交叉熵损失会随尺度改变**：把 w 放大会让所有 p 更极端，一旦有一个样本分错，损失就会爆炸。所以交叉熵**把尺度钉死在了使概率最贴合数据的那个值上**——这正是"校准"的数学含义。

推导 13：为什么初始损失恰好是 0.6931

参数全零时 $z = 0$，故 $p = \sigma(0) = 0.5$。代入交叉熵：

$$L = -[\,y \cdot \log(0.5) + (1-y) \cdot \log(0.5)\,] = -\log(0.5) \cdot [\,y + 1 - y\,] = -\log(0.5) = \mathbf{\log 2 \approx 0.6931}$$

注意 y 被完全消掉了——**不论标签是 0 还是 1，损失都是 ln 2**。所以整个数据集的平均损失也是 0.6931。

用途：0.6931 是二分类的**"零信息"基准线**。训练时若 log loss 停在 0.69 附近不动，说明模型什么都没学到。practical Q4B 从 0.6931 降到 0.3538，就是从"完全无知"往下走了一大步。

推导 14：边界的斜截式（含垂直线的特殊情况）

从 $w_1x_1 + w_2x_2 + b = 0$ 解 x_2：

$$w_2x_2 = -w_1x_1 - b \quad \rightarrow \quad x_2 = -(w_1/w_2)x_1 - b/w_2 \qquad (\text{要求 } w_2 \neq 0)$$

所以斜率 $m = -w_1/w_2$，纵截距 $c = -b/w_2$。

$w_2 = 0$ 的特殊情况：方程退化为 $w_1x_1 + b = 0$，即 $x_1 = -b/w_1$，这是一条**垂直线**。垂直线斜率无穷大，斜截式表示不了，必须单独处理。（若 w_1 也为 0，则方程变成 $b = 0$，边界要么是空集、要么是整个平面，模型退化。）

推导 15（拓展）：感知机收敛定理的错误上界

课程只要求知道"错误次数有限"，但定理其实给出了**具体的上界**，了解一下有助于理解"线性可分"到底意味着什么。

条件：存在单位向量 w^*（$\|w^*\| = 1$）使所有样本满足 $y_i(w^{*T}x_i) \geqslant \gamma > 0$（$\gamma$ 称为**间隔 margin**），且所有 $\|x_i\| \leqslant R$。

$$\text{感知机犯错次数} \leqslant (R / \gamma)^2$$

怎么读这个界：
- 分子 R 是数据的"尺度"，分母 γ 是两类之间的"缝隙宽度"。
- **缝隙越宽（γ 越大），需要的更新次数越少**；缝隙越窄，收敛越慢。
- **这个上界与样本数量 n 和维度 d 都无关**——这是感知机很漂亮的一个性质。
- 当数据不可分时 γ 不存在（可以理解为 $\gamma \leqslant 0$），上界失效，对应第 12 页说的"更新可能永不停止"。
- 顺带说明了 SVM 的动机：既然错误界是 $(R/\gamma)^2$，那么**直接去最大化 γ** 显然是个好主意——这就是最大间隔分类器。

推导速查：一条链从头到尾

步骤	式子	依据
1	$z = w^{\top}x + b$	模型定义
2	$p = \sigma(z) = 1/(1+e^{-z})$	sigmoid
3	$L = -[y \log p + (1-y) \log(1-p)]$	伯努利负对数似然（推导 6）
4	$\partial p / \partial z = p(1-p)$	推导 1
5	$\partial L / \partial p = (p-y)/[p(1-p)]$	推导 2
6	**$\partial L / \partial z = p - y$**	4 × 5，$p(1-p)$ 约掉
7	$\partial z / \partial w = x$，$\partial z / \partial b = 1$	线性函数求偏导
8	**$\partial L / \partial w = (p-y)x$，$\partial L / \partial b = p-y$**	6 × 7 链式法则
9	$\nabla_w J = (1/n)\Sigma(p_i - y_i)x_i$	对平均损失求导
10	**$w \leftarrow w - \eta \nabla_w J$**	梯度下降

附录 · 全模块速查表

项目	感知机	逻辑回归
得分	$z = w^Tx + b$（**共用**）	
输出变换	sign(z)	$\sigma(z) = 1/(1+e^{-z})$
标签	−1 / +1	0 / 1
损失	$\max(0, -y \cdot s)$	$-[y \log p + (1-y) \log(1-p)]$
损失范围	$[0, +\infty)$，线性增长	$[0, +\infty)$，对数爆炸增长
学习信号	$y \cdot s$（只看符号）	$p - y$（连续，带方向和大小）
权重更新	$w \leftarrow w + \eta y x$（加）	$w \leftarrow w - \eta(p-y)x$（减）
偏置更新	$b \leftarrow b + \eta y$	$b \leftarrow b - \eta(p-y)$
何时更新	仅 $y \cdot s \leqslant 0$	梯度非零即更新（几乎总是）
更新量大小	固定 $\eta y x$，不看错多少	按误差比例，接近正确时放缓
边界形状	**都是线性的**（$z = 0$），都解决不了 XOR	
概率	无	有，且校准
sklearn 方法	predict / decision_function	predict / decision_function / **predict_proba**

关键数值速记

$\sigma(0) = 0.5$	$\sigma(1) = 0.731$	$\sigma(2) = 0.881$	$\sigma(3) = 0.953$	$\sigma(4) = 0.982$	$\sigma(6) = 0.998$
$\sigma(-1) = 0.269$	$\sigma(-2) = 0.119$	$\sigma(-3) = 0.047$	$\sigma(-4) = 0.018$	$\sigma(-6) = 0.002$	$-\log(0.5) = 0.6931$

常用公式

- 几何距离 $= |z| / \|w\|$，$\|w\| = \sqrt{(w_1^2+w_2^2)}$
- 边界斜截式：$x_2 = -(w_1/w_2)x_1 - b/w_2$（斜率 $-w_1/w_2$，截距 $-b/w_2$，要求 $w_2 \neq 0$）
- 对称性：$\sigma(-z) = 1 - \sigma(z)$
- 逻辑回归梯度：$\partial L/\partial w = (p-y)x$，$\partial L/\partial b = p-y$
- 批量平均：$\partial J/\partial w = (1/m)\Sigma(p_i-y_i)x_i$